

Technical Report

AUTONOMOUS RC CAR

Prepared by: Tarek Sobih

Preparation for AVR Diploma
IMT School

Lecturer: Eng. Mostafa Mohamedy

2nd December 2024

Summary

This report discusses the application of the skills learned during the course through a practical project, resulting in the development of a working model of an autonomous car system.

Key findings include:

- **Design Phase:**
 - The car was designed in 3D using Autodesk Inventor.
 - A simulation of the steering system was created to ensure proper dimensions and functionality.
- **Execution Phase:**
 - The chassis was made from acrylic material, manufactured using CNC machining.
 - Steering components were 3D printed using PLA material.
- **Electronics Preparation:**
 - Necessary electronic components, such as motors, controllers, and sensors, were identified and purchased.
- **Electrical Circuit Design:**
 - The electrical circuit for the car was designed using Proteus software.
 - All connections were verified and tested in the simulation before physical implementation.
- **Assembly Phase:**
 - The mechanical and electrical components were assembled together.
 - All electrical connections were correctly wired.
- **Programming Phase:**
 - Necessary drivers for controlling the electrical components were prepared.
 - The core code for the car, including different driving modes, was written.
- **Results:**
 - The car can be controlled via Bluetooth from a smartphone.
 - The car operates in modes such as "Obstacle Avoidance" and "Line Following."

The information presented in this report was gathered from practical work and design efforts during the project.

This report has been prepared as part of the AVR Diploma.

TABLE OF CONTENTS

1. Summary	2
2. Introduction	4
3. Mechanical Design	4
4. Electrical & Electronic System	7
5. Programming	11
6. Implementation	15
7. Testing and Result	15
8. Challenges and Solutions	16
9. Recommendations	16
10. Conclusion	17

2.Introduction

This project focuses on the development of an autonomous RC car capable of following a predefined path using a line-following algorithm, avoiding obstacles, and allowing manual control via Bluetooth. The system is designed to seamlessly switch between autonomous and manual modes based on user preference.

Key Features:

1. **Obstacle Detection:** The car is equipped with an ultrasonic sensor to detect obstacles. When the car encounters an obstacle, it will stop or change direction based on the distance measured by the sensor, ensuring safe navigation.
2. **Bluetooth Control:** The car includes a Bluetooth module (e.g., HC-05), enabling manual control via a smartphone application. The user can remotely control the car's speed, direction, and toggle between autonomous and manual modes.
3. **Mode Switching (Autonomous vs. Manual):** The system allows users to switch between autonomous and manual modes. The transition between modes is indicated by a buzzer sound, providing feedback to the user that the mode has successfully changed.
4. **Motor Control:** The car uses DC motors for propulsion, with relays and transistors controlling motor operation. The steering is handled by a servo motor (optional for sharper turns). Pulse-width modulation (PWM) is employed for precise motor control, ensuring smooth adjustments to the car's speed.

This project demonstrates the integration of both hardware and software, applying key principles of autonomous control and wireless communication, while providing flexibility in operational modes for the user.

3. Mechanical Design

Case Study was used as a prototype for the project's mechanical design. The study included an initial design of the car chassis system, steering system, and motion system. After reviewing the basic design and analyzing the needs of the RC car project, major design modifications were made to ensure compatibility with the actual performance requirements.

OLD ONE

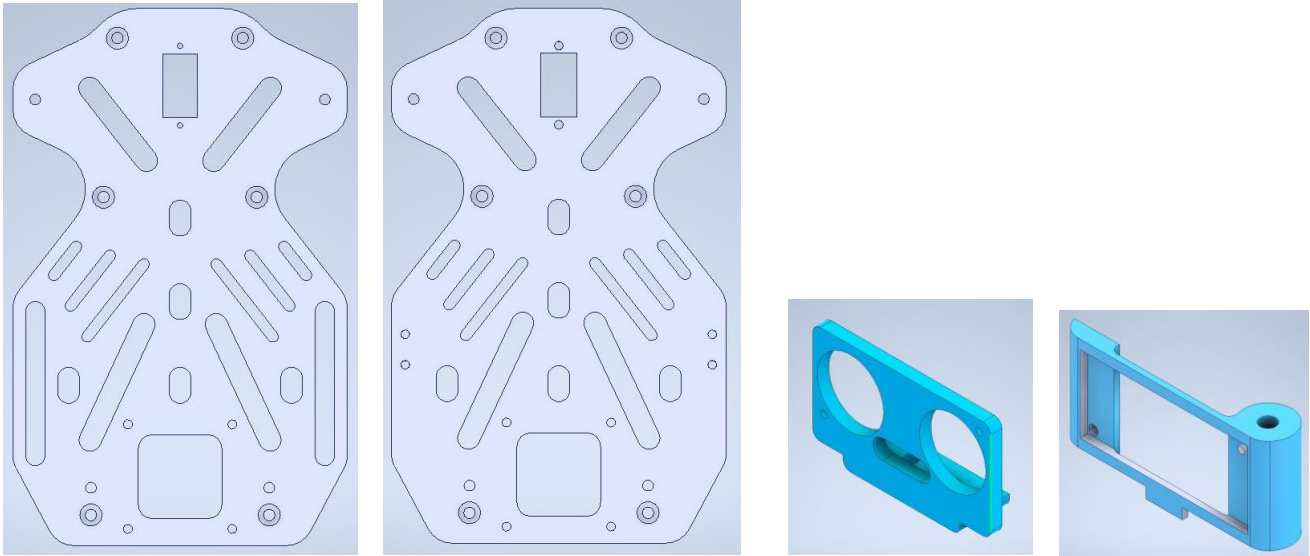


NEW ONE

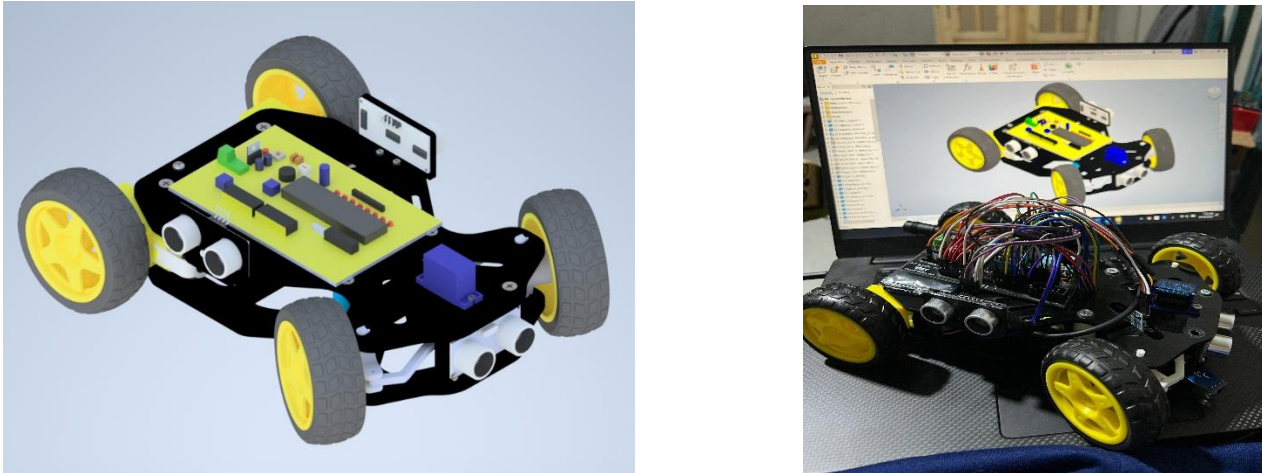


The design consist of chassis ,movement system with dc motors connected to rear wheels with 65mm diameter and steering system for front wheels with servo.

Edited parts:



There are a lot of hidden modification in dimensions due to allowance in printing and availability of mechanical component to be the final result like this.

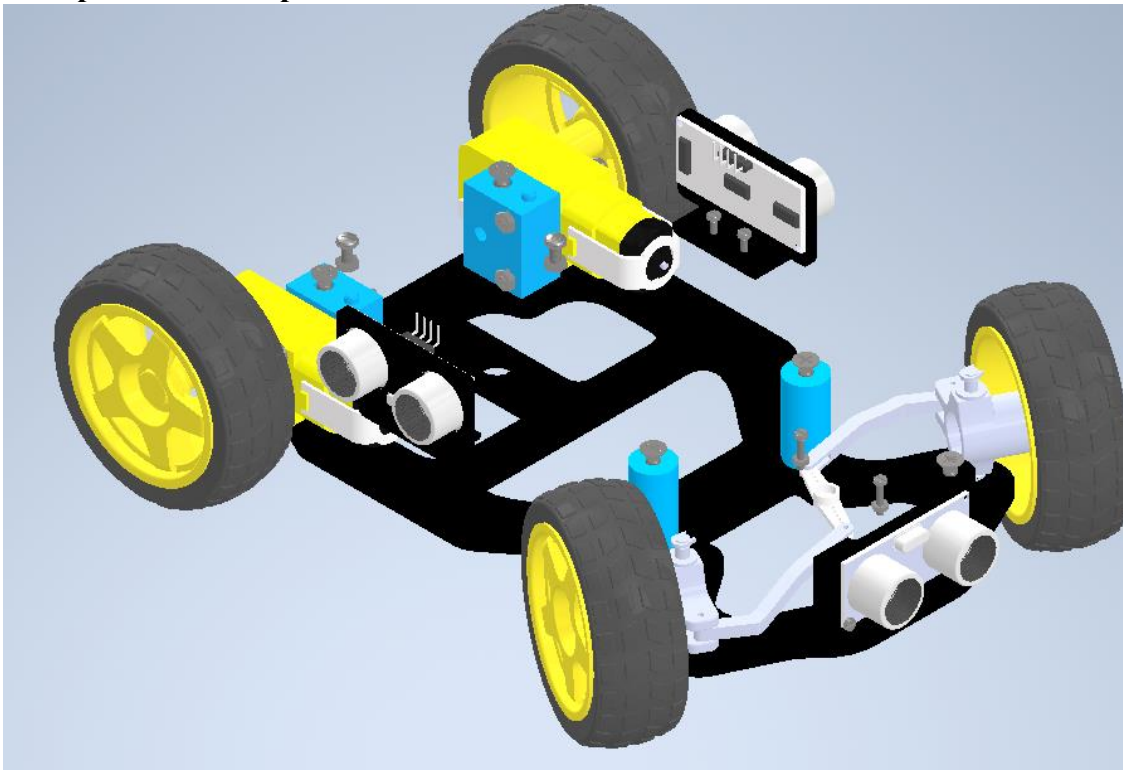


Chassis is made of acrylic to achieve strength and the light weight with low coast. Other components are 3D printed 100% solid.

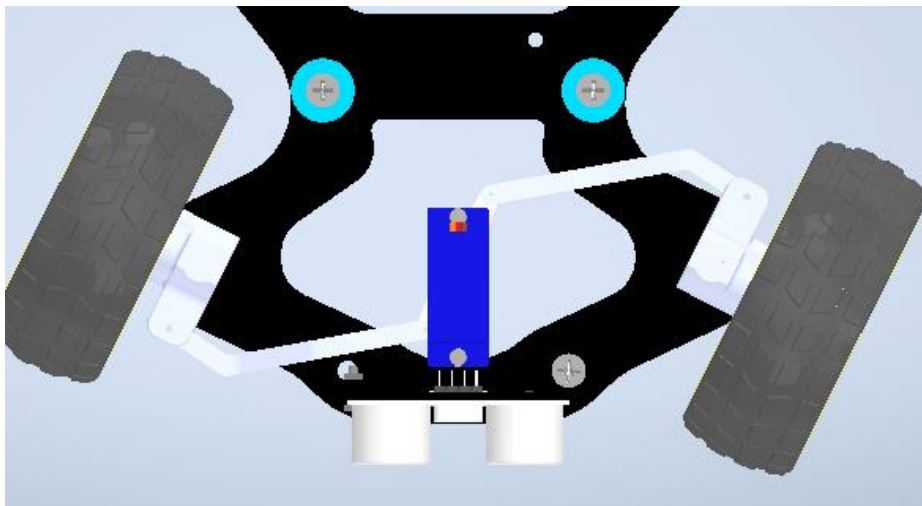
Why I used case study??

- ☐ Time and Effort Optimization.
- ☐ Proven and Reliable Design.
- ☐ Customization and Adaptability.
- ☐ Learning from Existing Expertise.
- ☐ Enhancing Performance with Modifications.
- ☐ Risk Reduction.
- ☐ Focus on Project Goals.

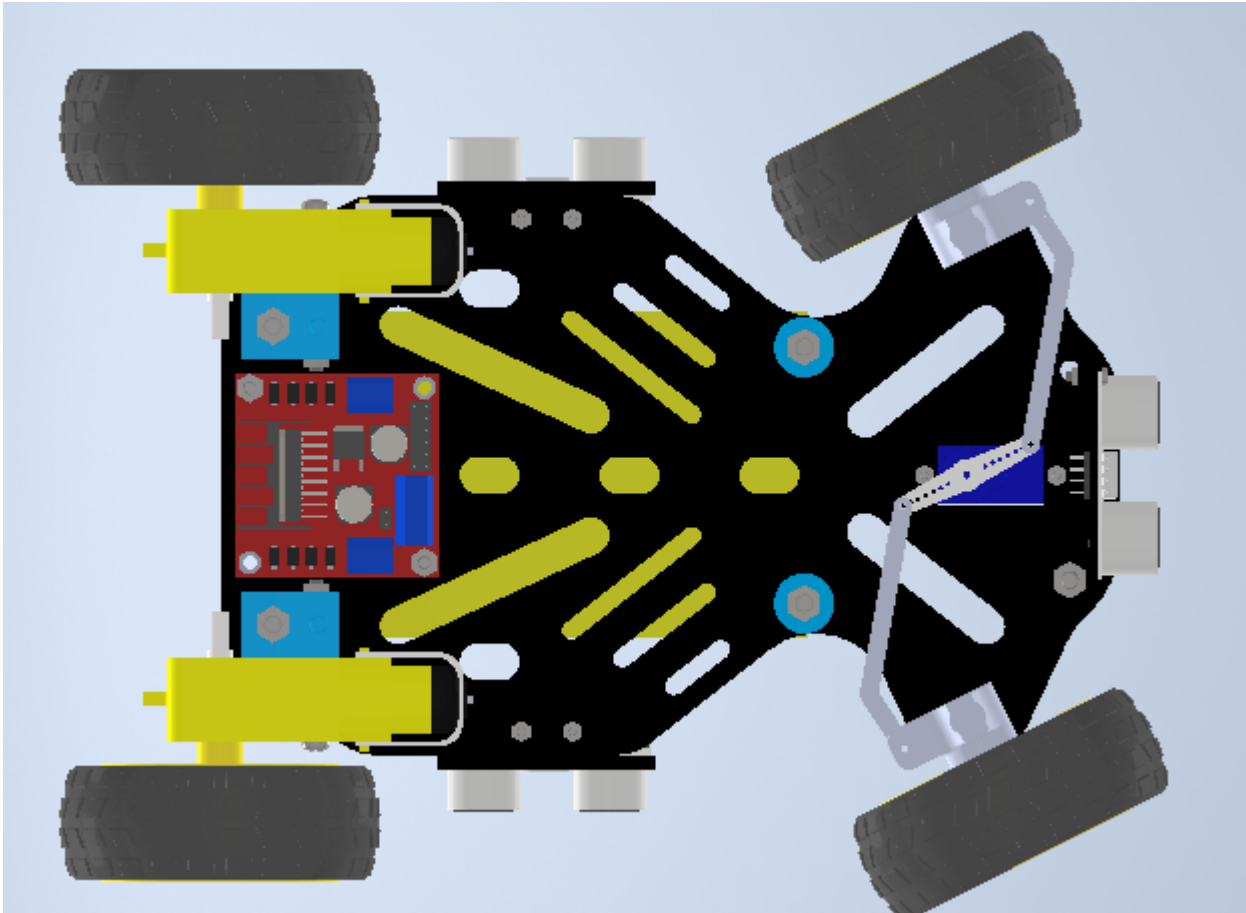
Some pictures of the parts:



DOWN CHASSIS



STEERING SYSTEM



UPPER CHASSIS

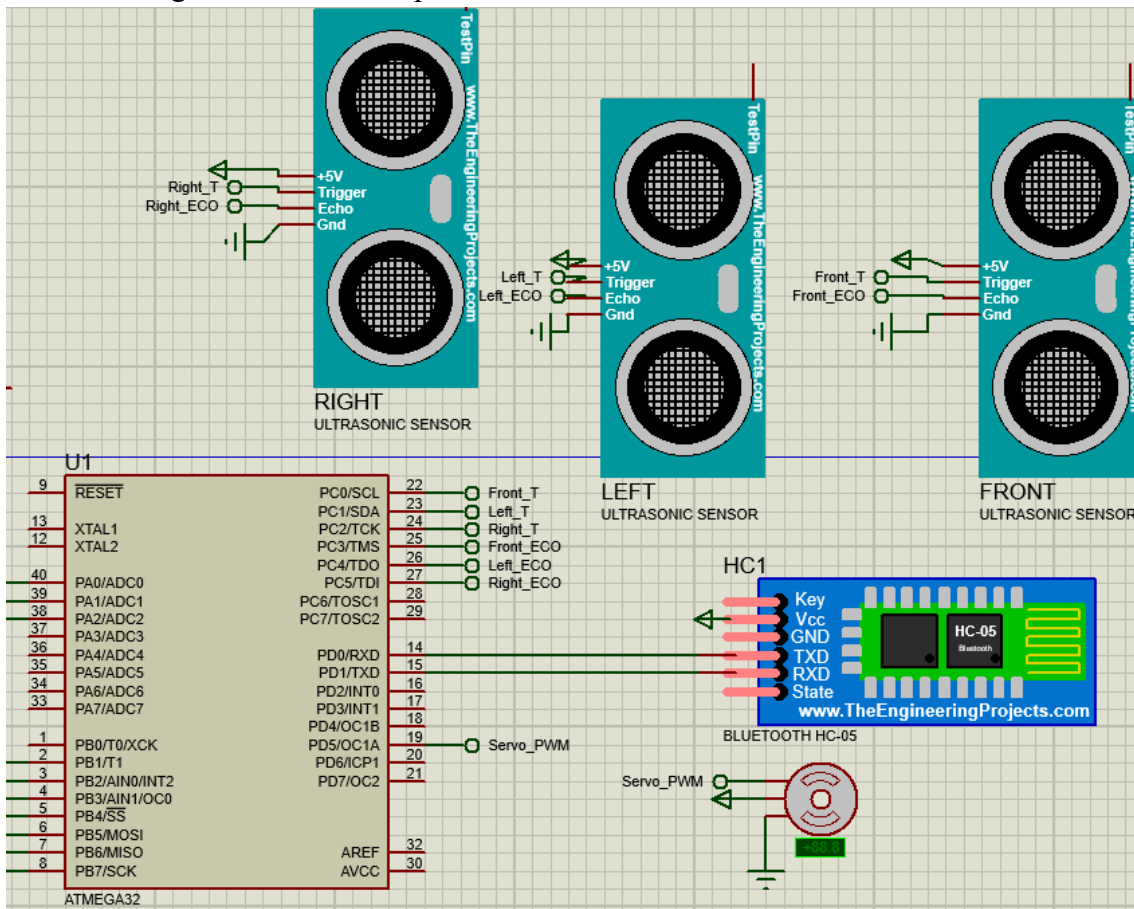
4. Electrical & Electronic System

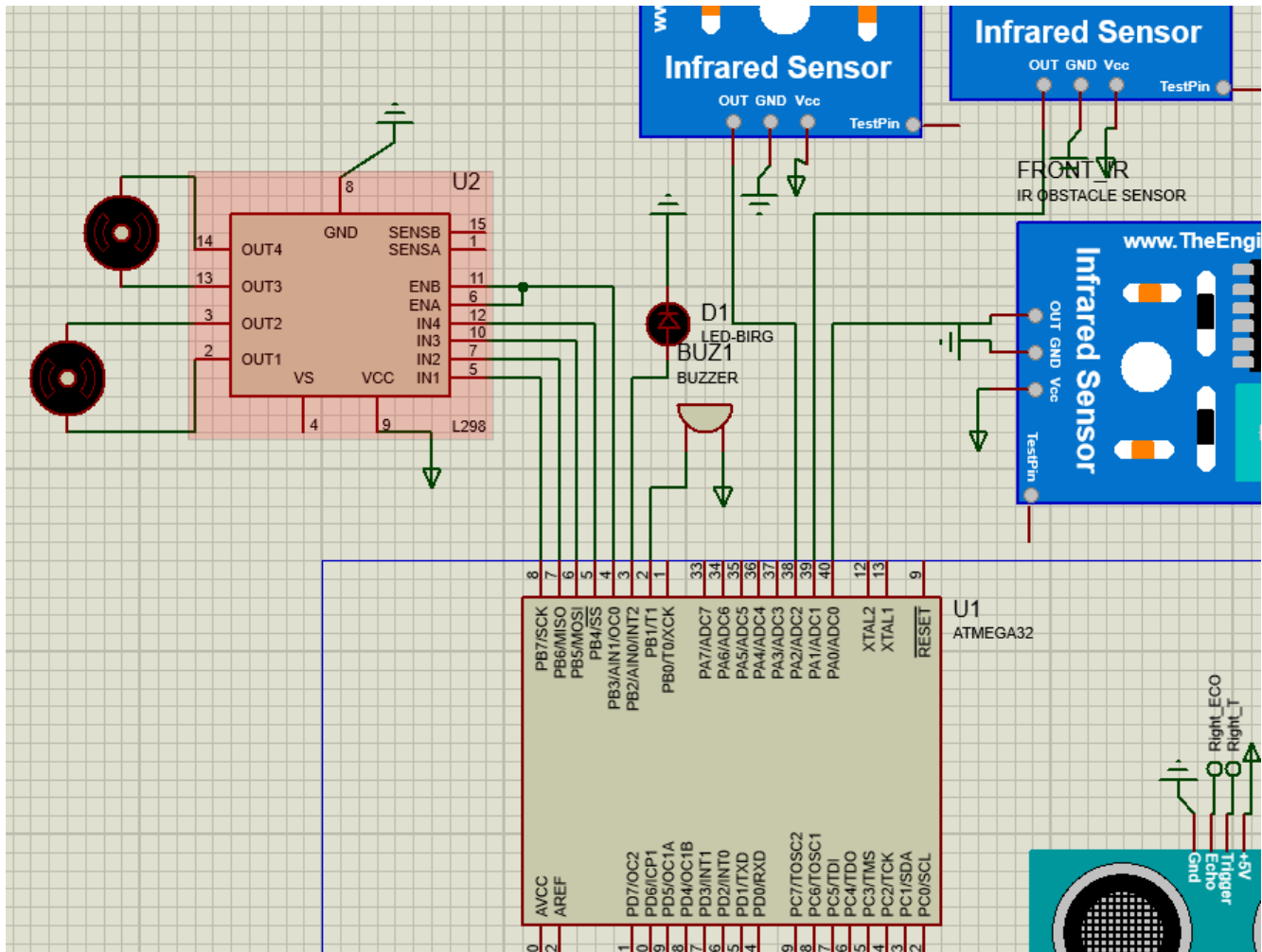
This section focuses on the design and implementation of the electrical circuits, components, and control systems that power and operate the car. The system includes essential components such as motors, sensors, microcontrollers, and motor drivers, all integrated to ensure smooth performance. This setup allows for precise control of the car's movements, responsiveness to inputs, and overall functionality. Through careful design and testing, the electric and electronic systems have been optimized to meet the project's requirements for efficiency, safety, and performance.

Component	Purpose/Usage
Microcontroller Atmega32	Controls the entire system, processes input from sensors, and sends commands to actuators.
DC Motors	Powers the movement of the RC car, enabling forward, backward, and turning actions.
Motor Driver (L298N)	Controls the direction and speed of the DC motors by receiving commands from the microcontroller.

Component	Purpose/Usage
Sensors (Ultrasonic)	Measures distance and helps with obstacle detection for safe movement.
Sensors (Infrared)	Indicate the black line to follow
Power Supply (Battery)	Provides electrical power to the microcontroller, motors, and other components.
BMS	Manage the charge and discharge of the batteries
LED (RGB)	Gives a visual effects
Buzzer	Used for visual feed back during operation
Servo Motor	Control the direction of the wheel
Bluetooth Module (HC-05)	Control the car manually ,send and receive data to MC

The circuit design for the electric and electronic system of the RC car was developed using Proteus. This design includes all essential components such as the microcontroller, motor driver, motors, sensors, and power supply, connected through a series of logical connections to ensure proper functionality. The schematic diagram below shows the connections and layout of the circuit, ensuring a clear understanding of how each component interacts to control the car's movements and other functionalities.





Power Management

Efficient power management is essential for the proper functioning of the RC car. The system is powered by a rechargeable battery 11.1V and 1000mAh , which provides energy to the microcontroller, motors, sensors, and other components.

The power distribution is managed through a central power distribution board that ensures that each component receives the necessary voltage and current.

The motors are powered by a motor driver that is directly connected to the battery, while the microcontroller and sensors are powered through a regulated power supply to ensure stable operation.

A voltage regulator is used to step down the battery voltage to a suitable level for the microcontroller and other low-power components. This careful management of power ensures that the system operates efficiently, preventing voltage drops or overloading, and extends the lifespan of the battery.

Control System (ATmega32)

The control system of the RC car is based on the ATmega32 microcontroller, which acts as the central processing unit for managing all operations. The ATmega32 is a powerful 8-bit microcontroller with sufficient processing power to handle multiple tasks simultaneously, such as controlling motor movement, processing sensor inputs, and managing communication protocols.

The microcontroller receives inputs from sensors such as the ultrasonic sensor, which detects obstacles, and processes them using programmed logic to control the behavior of the motors. The motor driver is interfaced with the ATmega32 to adjust the speed and direction of the DC motors based on the microcontroller's instructions.

Programming the ATmega32 was done in C language, and the microcontroller communicates with various components via GPIO (General Purpose Input/Output) pins. By using pulse width modulation (PWM) signals, the ATmega32 controls the speed of the motors for smooth and precise movements. The system ensures that all actions, from turning to obstacle avoidance, are executed in real-time with minimal delay.

Signal Processing

Signal processing is essential for interpreting data from sensors and ensuring smooth communication between the system components. The ultrasonic sensor used in this project emits sound waves, which bounce off objects and return as an echo. The time taken for the echo to return is measured and used to calculate the distance between the car and the object.

The **ATmega32 microcontroller** measures the duration of the echo pulse directly using its **GPIO pins**. The time difference between the emitted signal and received echo is then processed by the microcontroller to calculate the distance. This data is used to make real-time decisions, such as detecting obstacles and adjusting the car's movement accordingly, like stopping or changing direction to avoid collision.

Integration and Communication

Integration and communication between the components are vital for ensuring that all system parts work harmoniously. In this project, the **ATmega32 microcontroller** acts as the central controller, managing communication between sensors, the motor driver, and other system components.

To enable communication, the microcontroller uses **GPIO pins** to read input from sensors (such as ultrasonic or infrared) and send output to the motor driver for controlling the movement of the car. Additionally, for wireless communication with the **Bluetooth module**, the **ATmega32** uses the **UART (Universal Asynchronous Receiver/Transmitter)** protocol. This allows for bidirectional communication between the car and an external device, enabling remote control.

This integration ensures that signals from sensors are processed and actions are performed in real-time, providing an efficient and responsive control system for the RC car.

5. Programming

The programming of the RC car is the backbone of its functionality, enabling smooth operation and efficient control of all components. The code was written in **C language** and developed using Eclipse to ensure modularity and readability. The program was then uploaded to the **ATmega32 microcontroller** using a compatible programming tool.

The main functionalities implemented in the program include:

1. **Motor Control:** Code for controlling the speed and direction of the motors using **PWM (Pulse Width Modulation)** signals generated by the microcontroller.
2. **Sensor Integration:** Logic for reading inputs from the ultrasonic sensor and processing the data to detect obstacles and calculate distances.
3. **Bluetooth Communication:** Implementation of the **UART protocol** to enable wireless communication with the Bluetooth module for remote control commands.
4. **Decision-Making Algorithms:** Algorithms to interpret sensor data and make real-time decisions for obstacle avoidance or directional changes.
5. **InfraRed sensor :** Logic for reading inputs from this sensor and processing the data for line following.

Every part has its own drivers to separate functions for each task, then I combined these drivers in “[RC Car program.c](#)” to control the car.

I have made four main functions Init Func, manual mode func, obstacles func, line following func.

```
void RC_Car_voidInit(void)
{
    //Init UART pins
    DIO_voidSetPinDirection(UART_PORT , RX_PIN , DIO_u8INPUT);
    DIO_voidSetPinDirection(UART_PORT , TX_PIN , DIO_u8OUTPUT);

    //Init Led PIN
    DIO_voidSetPinDirection(DIO_u8PORTB , DIO_u8PIN2 , DIO_u8OUTPUT);

    //Init buzzer pin
    DIO_voidSetPinDirection(DIO_u8PORTB , DIO_u8PIN1 , DIO_u8OUTPUT);
    DIO_voidSetPinValue(DIO_u8PORTB , DIO_u8PIN1 , DIO_u8HIGH);

    SERVO_voidInit();    //Init the servo
    UART_voidInit();     //Init UART
    L298N_voidInit();    //Init Motor driver
}
```

Obstacles Mode Function:

```
void RC_Car_voidObstaclesMode(void)
{
    u8 Local_u8Switch;
    u8 Local_u8Speed = 8;
    u8 Buffer_State=0;

    f32 Local_f32FrontDistance;
    f32 Local_f32LeftDistance;
    f32 Local_f32RightDistance;

    //Set motor speed to suitable speed
    L298N_voidSetSpeed(Local_u8Speed);

    //Set Wheels position
    SERVO_voidSetPosition(STRAIGHT_ANGLE);

    //Init ultrasonic sensors
    ULTRASONIC_voidInit();

    while(Global_u8Mode == 1)
    {
        //Check if there new data in UART buffer
        Buffer_State = UART_u8CheckForData();

        if (Buffer_State == 1)
        {
            //get input from Smart Phone
            Local_u8Switch = UART_u8ReceiveData();

            // To modify speed according to input
            if(Local_u8Switch >= '0' && Local_u8Switch <= '9')
            {
                //convert from "ASCII" to "int"
                Local_u8Speed = Local_u8Switch - '0';
                L298N_voidSetSpeed(Local_u8Speed);
            }
            else
            {
                switch (Local_u8Switch)
                {
                    case TOP_SPEED :           //to set speed to max
                        L298N_voidSetSpeed(MAX_SPEED);
                        break;
                    case MANUAL_MODE:           //to exit the mode
                        UART_voidSendString("Exit obstacles mode");
                        Global_u8Mode = 0;
                        break;
                    case LINE_FOLLOWING_MODE:    //to exit the mode
                        UART_voidSendString("Exit obstacles mode");
                        Global_u8Mode = 2;
                        break;
                    default:
                        break;
                }
            }
        }

        //get front distance
        Local_f32FrontDistance = ULTRASONIC_f32GetFrontDistance();
        _delay_ms(20); //wait sensor to read

        if (Local_f32FrontDistance > MIN_DISTANCE) //minimum distance is 10cm
        {
            L298N_voidMoveForward();
            while (Local_f32FrontDistance > MIN_DISTANCE)
            {
                //update sensor reading while moving
                Local_f32FrontDistance = ULTRASONIC_f32GetFrontDistance();
                _delay_ms(20);
            }
            L298N_voidBrake();
            UART_voidSendData('!'); //test Character
        }
        else
        {
            L298N_voidBrake();

            //get left distance
            Local_f32LeftDistance = ULTRASONIC_f32GetLeftDistance();
            _delay_ms(20); //wait sensor to read

            //get right distance
            Local_f32RightDistance = ULTRASONIC_f32GetRightDistance();
            _delay_ms(20); //wait sensor to read

            while(Local_f32FrontDistance < SAFE_DISTANCE)
            {
                L298N_voidMoveBackward();
                Local_f32FrontDistance = ULTRASONIC_f32GetFrontDistance();
                _delay_ms(20);
            }
            L298N_voidBrake();
            UART_voidSendData('@'); //test character

            if(Local_f32LeftDistance > Local_f32RightDistance)
            {
                //To turn the wheels left
                L298N_voidTurnLeft();
            }
            else
            {
                //to turn the wheels to right
                L298N_voidTurnRight();
            }
            UART_voidSendData('#'); //test character
        }
    }
}
```

Manual Mode Function:

```

void RC_Car_voidManualMode(void)
{
    u8 Local_u8Switch;
    u8 Local_u8Speed = 5;

    //Set speed to suitable speed
    L298N_voidSetSpeed(Local_u8Speed);

    //set position of the wheels
    SERVO_voidSetPosition(STRAIGHT_ANGLE);

    while (Global_u8Mode == 0)
    {
        //Read data from smart phone
        Local_u8Switch = UART_u8ReceiveData();

        // if condition for speed variation
        if(Local_u8Switch >= '0' && Local_u8Switch <= '9')
        {
            //convert from "ASCII" to int
            Local_u8Speed = Local_u8Switch - '0';
            L298N_voidSetSpeed(Local_u8Speed);
        }
        else
        {
            switch (Local_u8Switch)
            {
                case 'q':
                    L298N_voidSetSpeed(10);
                    break;
                case 'F': //Moving Forward
                    SERVO_voidSetPosition(STRAIGHT_ANGLE);
                    L298N_voidSetSpeed(Local_u8Speed);
                    L298N_voidMoveForward();
                    break;
                case 'B': //Moving backward
                    SERVO_voidSetPosition(65);
                    L298N_voidSetSpeed(Local_u8Speed);
                    L298N_voidMoveBackward();
                    break;
                case 'R': //turn wheels right
                    L298N_voidSetSpeed(0);
                    SERVO_voidSetPosition(110);
                    break;
                case 'L': //turn wheels left
                    L298N_voidSetSpeed(0);
                    SERVO_voidSetPosition(20);
                    break;
                case 'G': //moving left forward
                    L298N_voidSetSpeed(Local_u8Speed);
                    SERVO_voidSetPosition(LEFT_TURN_ANGLE);
                    L298N_voidMoveForward();
                    break;
                case 'I': //moving right forward
                    L298N_voidSetSpeed(Local_u8Speed);
                    SERVO_voidSetPosition(RIGHT_TURN_ANGLE);
                    L298N_voidMoveForward();
                    break;
                case 'H': //moving left backward
                    L298N_voidSetSpeed(Local_u8Speed);
                    SERVO_voidSetPosition(LEFT_TURN_ANGLE);
                    L298N_voidMoveBackward();
                    break;
                case 'J': //moving right backward
                    L298N_voidSetSpeed(Local_u8Speed);
                    SERVO_voidSetPosition(RIGHT_TURN_ANGLE);
                    L298N_voidMoveBackward();
                    break;
                case 'S': //stop and return wheels straight
                    SERVO_voidSetPosition(STRAIGHT_ANGLE);
                    L298N_voidBrake();
                    break;
                case 'W': //Led on
                    DIO_voidSetPinValue(DIO_u8PORTB , DIO_u8PIN2 , DIO_u8HIGH);
                    break;
                case 'w': //led off
                    DIO_voidSetPinValue(DIO_u8PORTB , DIO_u8PIN2 , DIO_u8LOW);
                    break;
                case 'V': //buzzer on
                    DIO_voidSetPinValue(DIO_u8PORTB , DIO_u8PIN1 , DIO_u8LOW);
                    break;
                case 'v': //buzzer off
                    DIO_voidSetPinValue(DIO_u8PORTB , DIO_u8PIN1 , DIO_u8HIGH);
                    break;
                default: //to exit the mode
                    if (Local_u8Switch == 'N') Global_u8Mode = 1;
                    if (Local_u8Switch == '@') Global_u8Mode = 2;
                    break;
            }
        }
    }
}

```

Line Following Mode Function:

```
void RC_Car_voidLineFollowingMode(void)
{
    u8 Local_u8Switch;
    u8 Local_u8Speed = 5;

    u8 Local_u8Middle;
    u8 Local_u8Left;
    u8 Local_u8Right;

    u8 CompinedReading;
    //Set motor speed to suitable speed
    L298N_voidSetSpeed(Local_u8Speed);

    //Set Wheels position
    SERVO_voidSetPosition(STRAIGHT_ANGLE);

    //init ir sensors
    InfraRed_voidInit();

    while(Global_u8Mode == 2)
    {
        //get input from Smart Phone
        Local_u8Switch = UART_u8ReceiveData();

        // To modify speed according to input
        if(Local_u8Switch >= '0' && Local_u8Switch <= '9')
        {
            //convert from "ASCII" to "int"
            Local_u8Speed = Local_u8Switch - '0';
            L298N_voidSetSpeed(Local_u8Speed);
        }
        else
        {
            switch (Local_u8Switch)
            {
                case TOP_SPEED: //to set speed to max
                    L298N_voidSetSpeed(MAX_SPEED);
                    break;
                case MANUAL_MODE: //to exit the mode
                    UART_voidSendString("Exit line following mode");
                    Global_u8Mode = 0;
                    break;
                case OBSTCALES_MODE: //to exit the mode
                    UART_voidSendString("Exit line following mode");
                    Global_u8Mode = 2;
                    break;
                default:
                    break;
            }
        }
        //Get sensors readings
        Local_u8Middle = InfraRed_u8GetFrontRead();
        Local_u8Left = InfraRed_u8GetLeftRead();
        Local_u8Right = InfraRed_u8GetRightRead();

        //Compine values in one pinary number
        CompinedReading = (Local_u8Left << 2) | (Local_u8Middle << 1) | Local_u8Right;

        switch (CompinedReading)
        {
            case 000:
                break;
            case 010:
                break;
            case 110:
                break;
            case 011:
                break;
            case 100:
                break;
            case 001:
                break;
            case 101:
                break;
            case 111:
                break;
            default:
                break;
        }
    }
}
```

6. Implementation

The implementation of the RC car involved the following steps:

1. Mechanical Assembly:
 - Assembling the chassis, wheels, and other mechanical components to form the base of the car.
 - Securing the motor mounts and ensuring proper alignment of the drivetrain.
2. Electronic Setup:
 - Connecting the ATmega32 microcontroller to the sensors, motor driver, and Bluetooth module.
 - Wiring the power supply to distribute power efficiently to all components.
3. Programming and Uploading:
 - Writing the control code in C language using Eclipse.
 - Uploading the code to the microcontroller using a usbasp.
4. Preliminary Testing:
 - Verifying the connections using a multimeter.
 - Testing individual components, such as motors, sensors, and communication with the Bluetooth module, to ensure they function as expected.
5. System Integration:
 - Combining all components (mechanical and electronic) into a single system.
 - Running the system to ensure all parts work harmoniously.
6. Debugging and Optimization:
 - Identifying and resolving any issues in the hardware or software for every mode respectively.
 - Making adjustments to improve the car's performance, such as fine-tuning motor speeds or sensor thresholds.
7. Final Testing:
 - Conducting full system tests in a real-world environment to verify functionality.
 - Ensuring the car responds correctly to remote control commands and obstacle detection and line following.

7. Testing and Result

Manual Mode : I have tested every thing in this mode and it works perfectly.

Obstacle Mode : every thing tested and work perfectly.

Line Following Mode : I have made 90% from the code but I didn't test it because IR sensors doesn't work.

8. Challenges and Solutions

1. **Challenge:** Difficulty in connecting the steering system components due to mismatched pin diameters.
 - **Solution:** Enlarged the holes to accommodate slightly larger screws, ensuring a secure connection.
2. **Challenge:** No provisions were made in the design for mounting the infrared sensors.
 - **Solution:** Found an alternative solution and mounted the sensors securely and effectively.
3. **Challenge:** The purchased servo motor turned out to be a continuous servo instead of a regular one.
 - **Solution:** Replaced it with a servo motor from IMT kit, soldered wires to it, and integrated it into the system.
4. **Challenge:** Limited GND and VCC pins.
 - **Solution:** Created a common GND and VCC connection for the sensors, resolving the issue.
5. **Challenge:** Difficulty in removing the batteries for recharging.
 - **Solution:** Added a suitable BMS (Battery Management System) circuit to the car, allowing the batteries to be charged in place without removal.
6. **Challenge:** Struggled with writing the ultrasonic sensor driver during programming.
 - **Solution:** Spent a week debugging and eventually resolved the issue, achieving excellent performance.
7. **Challenge:** The steering system required a 25 cm clearance in front of the car to turn left or right.
 - **Solution:** Modified the programming to optimize turning, enabling the car to maneuver in tighter spaces.
8. **Challenge:** Infrared sensor issues in differentiating between black and white colors.
 - **Solution:** The issue remains unresolved as of now, and further work is needed to refine the sensor readings.
9. **Challenge:** Other minor coding issues encountered during development.
 - **Solution:** Detected and resolved these issues through rigorous debugging and testing.

9. Recommendations

To enhance the RC car project, the following recommendations are proposed to improve functionality, accuracy, and user experience:

1. Add a Rear Sensor:
 - Implement a rear ultrasonic sensor to assist with obstacle detection when the car moves backward.
2. Incorporate a Gyroscope:
 - Use a gyroscope to measure the car's drift angle and provide accurate feedback to the servo motor for better precision in turning.
3. Software Improvements:
 - Optimize the code for better efficiency and accuracy.
 - Utilize FreeRTOS to manage multiple tasks more effectively.

4. Mechanical Design Enhancements:
 - Design future models with dedicated spaces for mounting additional components like sensors or modules.
 - Improve the car's structure to handle various environments and terrains.
5. Advanced Features:
 - Integrate artificial intelligence for advanced obstacle avoidance or line-following capabilities.
 - Add an LCD or OLED screen to display real-time data such as current speed, mode, and obstacles encountered.
6. Brake System:
 - Introduce a braking system for the rear wheels to enhance stopping efficiency at higher speeds.
7. Electronic Component Upgrades:
 - Select higher-quality electronic components in future iterations to ensure precise data and better reliability.
8. Battery Monitoring:
 - Implement a notification system to alert the user when the battery is low, enhancing user convenience.
9. Watchdog Timer:
 - Use a watchdog timer to reset the microcontroller in case of any system errors, improving reliability and fault tolerance.

By incorporating these recommendations, the RC car system will become more versatile, efficient, and capable of addressing various challenges while offering a better user experience.

10. Conclusion

This project was part of my graduation requirement for the AVR Embedded Systems course, where I designed and implemented a smart RC car capable of autonomous movement and obstacle avoidance. The mechanical design was carefully selected to ensure stability, while the electronic and electrical systems were integrated to work seamlessly with sensors like ultrasonic and infrared for real-time decision-making.

The ATmega32 microcontroller served as the heart of the system, managing communication between components and processing sensor data. The software was developed to control the car's movement, process sensor data, and react to obstacles in the environment.

The project was successfully completed with satisfactory results. Future improvements may include adding more sensors, enhancing the software algorithms, and upgrading the mechanical design to improve efficiency and adaptability.

This project provides a strong foundation for further exploration into autonomous systems and robotics, building upon the skills and knowledge gained during the AVR Embedded Systems course.

Best wishes,
Tarek Sobih